

TimeRLM: Recursive Language Models Enable Precise Anomaly Localization in Long-Context Time-Series

Nicolas Zumarraga^{1*} Lorenzo Steno^{1,3} Ning Wang¹ Max Rosenblatt² Thomas Kaar²
 Maxwell A. Xu^{4,5} Kevin O’Sullivan¹ Markus Kreft¹ Elgar Fleisch^{1,6,7} Paul Schmiedmayer²
 Patrick Langer^{1,2†} Robert Jakob^{1†}

¹Agentic Systems Lab, ETH Zürich ²Stanford University ³Seldon Technologies ⁴University of Illinois Urbana-Champaign ⁵Google
⁶Centre for Digital Health Interventions, ETH Zürich ⁷Centre for Digital Health Interventions, University of St. Gallen

Abstract

Precise anomaly localization over long-context time-series is a crucial task in monitoring applications across clinical care, industrial operations, financial services, and logistics, where brief evidence may hide inside long spans of high-frequency data. Time Series Language Models (TSLMs) are able to ingest time series data and verbalize findings on anomalies in natural language; however recent benchmarks report a decrease in retrieval performance at long time horizons, mirroring failure modes for long-context retrieval in text, vision, and audio. In the text domain, Recursive Language Models (RLMs) can recover much of this lost performance by keeping context external to the Large Language Model (LLM) rather than placing it in the input prompt, allowing the model to query and manipulate it through code. We present **TimeRLM**, an RLM formulation for time-series that sequentially manipulates the signal using code and vision capabilities. To assess performance, we further introduce **AnomalyXL**, a synthetic long-context anomaly localization benchmark with programmatically injected anomalies that require precise retrieval. We implement five different task categories and two variants; AnomalyXL-MCQ and AnomalyXL-Localize. TimeRLM outperforms every evaluated TSLM and frontier single-pass model on four of the five AnomalyXL-Localize tasks, reaching 0.682 Intersection over Union (IoU) on localization and 0.745 on classify-with-evidence, compared with at most 0.329 and 0.072 across all baselines. To further improve performance, we train TimeRLM using reinforcement learning and re-evaluate it on the same datasets. The resulting model further improves performance and outperforms TSLMs on precise retrieval tasks while using an LLM backbone up to eight times smaller. It also requires approximately one-third as many agent interaction turns as the untrained base model to produce a final answer. When evaluated on previously unseen real-world Electrocardiogram (ECG), sleep and software observability recordings, the post-trained TimeRLM retains or improves performance, surpassing TSLMs despite being trained exclusively on synthetic data. Our findings suggest recursive interaction with time-series as an effective approach for long-horizon temporal reasoning and retrieval.

1 Introduction

In healthcare, manufacturing, software operations, financial services, and logistics, sensors routinely produce long continuous data streams (Yèche et al. 2021; Yuan et al. 2024). These streams may span hours of intensive-care vital signs (Yèche et al. 2021), days of wearable activity (Yuan et al. 2024; Chan et al. 2024), or years of energy telemetry (Kelly and Knottenbelt 2015). Yet the events that matter within this data are often brief anomalies—such as a few abnormal beats, a transient arousal, or a short latency spike—buried in otherwise conventional signals. Identifying these events requires models that can efficiently scan long time-series, localize relevant intervals, and explain why they are unusual (Su et al. 2024). Early work applied Large Language Models (LLMs) to anomaly detection and reasoning, but LLMs struggle to interpret time-series data (Langer et al. 2026). Recent advances in multimodal modeling have enabled Time Series Language Models (TSLMs) to process raw time-series as a native modality, making them a promising approach for detecting anomalies and describing their characteristics in natural language (Zumarraga et al. 2026; Wang et al. 2025a; Langer et al. 2026).

Different TSLM architectures have been proposed to fuse time-series, for example using full-resolution patch adapters (Wang et al. 2025a; Xie et al. 2025), compression of time-series into fixed latent vectors (Langer et al. 2026; Wang et al. 2025b), and time-series–vision hybrids using rendered plots (Xie et al. 2026). While architecturally different, all ingest time-series into context and answer in a single forward pass: the recording is encoded once, at fixed or compressed size, and the model answers from this single view. In evaluation, time-series question-answering benchmarks have measured model performance on short context anomaly classification, or reduced long horizon localization tasks to multiple-choice questions (Cai et al. 2024; Xie et al. 2026), side-stepping the precise temporal retrieval that long, real-world signals require. Recent evaluations on long-context TSLM retrieval have exposed performance limitations of single-pass TSLMs, where precise retrieval performance collapses at increased time-series lengths, leading to TSLMs missing fine-grained events or anomalies (Zumarraga et al. 2026). This finding aligns with similar long-context retrieval failure modes in text (Kamradt 2023; Liu et al. 2024; Kuratov et al. 2024), vision (Wang et al. 2024a;

  — <https://github.com/OpenTSLM/TimeRLM>

*Corresponding author – nzumarraga@ethz.ch

†Equal contribution as senior authors.

Question: Is there an anomaly? If so, of which type and where?

Ground truth: Hypoapnea – start: 04:12 | end : 04:20

Recursive Language Model (RLM)

The question appears to be related to a **sleep recording**. Let me verify the time-series length, number of channels and channel names. `<CODE>`

`<REPL output appended to context chain>`

The signal spans 2 minutes of recording and 13 channels. Sleep anomalies are usually characterized by sudden respiratory obstructions. I will call a subagent to identify anomalous candidates on the four most relevant channels to capture these anomalies: FLOW, PRES, THO and ABD. `<CODE>`

Sub-RLM

Let me investigate the signal further... `<CODE>` On the four channels, I will calculate rolling mean, deviation and sudden drops and perform a spectral decomposition of the signal... `<CODE>`

Answer: Potential respiratory anomaly between 04:12 and 04:20

Good findings, now call a **second subagent** to classify the respiratory anomaly at that precise location for these channels. `<CODE>`

Answer: Hypoapnea – start: 04:12 | end : 04:20

REPL Environment

Python execution runtime

```

>> sub_rlm (signal = signal[:200],
channels = {'FLOW', 'PRES', 'THO',
'ABD'}, question = q)

>> for b in rolling_windows:
m, d = measure_stats(b)
print(f"Mean {m}, dev: {d}")

>> plt.plot([x], y, data=s[10:400])
>> look(plt.show())

```

Environment Variables

signal → time-series data
sub_rlm() → callable rlm
look() → view plot
FINAL() → submit answer

Packages

import numpy
import scipy
import matplotlib

Image-mode add. package

■ Apnea
■ N2
■ Sampled segments

Figure 1: TimeRLM interacts with time-series through code. The orchestrator reasons in natural language and emits Python into a sandboxed REPL (right), where the signal is an environment variable. Sub-RLMs can be invoked.

Dingjie et al. 2024), and audio (He et al. 2025).

Recent agentic search paradigms employ tool use (Yao et al. 2023; Wang et al. 2024b) and RL-trained search (Jin et al. 2025; Chen et al. 2026) to improve performance by letting the model interact with the input over multiple turns instead of ingesting context in a single pass. To that end, Recursive Language Models (RLMs) store the full content as a variable rather than placing it in the model’s context window, allowing it to query and manipulate it through code (Zhang, Kraska, and Khattab 2026). Recent work in agentic time-series reasoning has proposed analyzing signals through similar approaches, using tool calls and learned segment selection (Zhao et al. 2025; Messica et al. 2026), showcasing the potential of avoiding full context ingestion for time-series reasoning. However, a general, training-free formulation with direct access to the raw signal that works over long-context data is yet to be proposed for time-series reasoning. We therefore introduce **TimeRLM**, an RLM formulation that enables manipulating time-series signals as external context, querying them over multiple turns in a sandboxed Python environment, and reasoning through code and vision to submit a structured answer grounded in the underlying signal. To evaluate TimeRLM, we additionally introduce **AnomalyXL**, an open-source benchmark measuring precise anomaly localization in long horizon time-series spanning two evaluation corpora, **AnomalyXL-MCQ** and **AnomalyXL-Localize**. AnomalyXL is, to our knowledge, the first synthetic long-context anomaly benchmark released together with code implementations for the generator and Reinforcement Learning (RL) environment. We provide configurable difficulty for generating training and evaluation data and tasks. In addition to evaluating on AnomalyXL, we further evaluate TimeRLM performance on the anomaly-property tier of ARFBench (Xie et al. 2026), a multiple-

choice benchmark built from real production telemetry. We also assess zero-shot transfer to a generalization set built from real Electrocardiogram (ECG) (LTAF dataset) (Petrucci, Sahakian, and Swiryn 2007) and sleep (Ghassemi et al. 2018) recordings (Sleep-PSG), and explore post-training TimeRLM. On average, TimeRLM improves upon all evaluated TSLM baselines on all evaluated benchmarks, and transfers zero-shot to unseen clinical domains where post-training on synthetic data retains the model’s capabilities. We release all code and data for both TimeRLM and AnomalyXL fully open-source.

2 Related Work

In the following we explore related work to TimeRLM among TSLMs, anomaly detection benchmarks, and agentic methods for long context retrieval.

Time-Series Language and Foundation Models. TSLMs are multimodal LLMs that incorporate time-series as a native modality (Langer et al. 2026). TSLMs differ in how each model processes time-series of length L . *Full-resolution adapters* pass raw tokens or $N = L/P$ patch embeddings directly to the LLM, preserving signal fidelity but leading to quadratic compute and memory growth due to $\mathcal{O}(N^2)$ cost of self-attention (Wang et al. 2025a; Xie et al. 2025; Vaswani et al. 2017). A different approach based on *fixed-latent compression* places a module between the encoder and the LLM that distills the signal into a constant number of tokens. ITFormer’s summary tokens (Wang et al. 2025b) and OpenTSLM-Flamingo’s Perceiver Resampler (Langer et al. 2026; Jaegle et al. 2021) allow computational cost to remain almost constant at the LLM level; however the compression ratio grows with longer time-series, compressing more data into the same



Figure 2: AnomalyXL: evidence-grounded anomaly localization. Five task families (localize, classify, measure magnitude, localize in all channels, lead-lag) in two formulations over the *same* signal: AnomalyXL-MCQ and AnomalyXL-Localize.

number of fixed latent tokens. A more recent line of work pairs inputs from a frozen time-series foundation model and rendered time-series plots with a Vision Language Model (VLM) backbone, using both modality tokens to reason over the time-series signal (Xie et al. 2026).

Anomaly Detection and Short-Context Prediction. Time-series anomaly detection is traditionally formulated as identifying anomalous points or subsequences, often by assigning anomaly scores to sliding-window segments (Boniol et al. 2024). With the advent of LLMs, a new paradigm of reasoning over signals in natural language to explain why specific segments are anomalous has emerged (Su et al. 2024). Benchmarks in this field typically evaluate anomaly detection accuracy over short, pre-segmented recordings, using semantic labels (Oh et al. 2023) and Multiple Choice Question (MCQ) answering formats (Xie et al. 2026).

Recursive Language Models and Long-Context Agents. Agentic frameworks decompose complex tasks into a series of LLM reasoning steps and actions, leveraging tool use (Yao et al. 2023) and iterative retrieval (Wang et al. 2024b). RLMs take this concept further by treating context as an environment variable that a *root* LLM queries through code and which can recursively spawn *sub-RLMs* on segments of this context, rather than ingesting it directly (Zhang, Kraska, and Khattab 2026). In time-series, recent work shows how LLMs are capable of time-series analysis using code (Xu et al. 2026). TimeSeriesScientist and ARTS orchestrate specialized forecasting and classification tools respectively, decoupling task-specific proficiency from global reasoning (Zhao et al. 2025; Zumarraga et al. 2026). ARTIST learns segment selection and time-series reasoning with reinforcement learning (Messica et al. 2026).

3 Methods

In the following, we present details on TimeRLM in Section 3.1 and AnomalyXL in Section 3.2. TimeRLM is an RLM formulation that enables reasoning over uncompressed multivariate time-series using code, whereas AnomalyXL is a synthetic long-context time-series benchmark. We additionally introduce a generalization set, sampled from real ECG and Sleep clinical recordings and explore post-training for the RLM formulation.

3.1 TimeRLM Formulation

We formulate TimeRLM as a Recursive Language Model (Zhang, Kraska, and Khattab 2026) where the time-series is queryable from a context file. We start with a text-only RLM. Over multiple **turns** (i.e., RLM agent actions), the model can emit Python into a code sandbox (i.e., REPL) with access to standard python data science libraries, receiving each execution’s REPL output in its context. A JSON object containing the time-series is pre-defined as a variable. This setup allows the model to operate over the full recording directly through code (Fig. 1). We also explore a second RLM setup in which the sandbox also provides access to python vision libraries, allowing the model to render time-series as a plot and perceive it visually. In a third setup, we allow the model to recursively call instances of itself (*sub-RLMs*) on the whole or a fraction of the time-series.

Anomaly Localization as a Decision Process. We formalize precise anomaly localization as a finite-horizon sequential decision process. Given a recording $\mathbf{X} \in \mathbb{R}^{C \times L}$ over C channels of length L and a question q , an LLM denoted as policy π_θ (the root LLM) produces an answer $\hat{y}$ over a horizon of N interactions ($t = 1, \dots, N$). At step t

the state $s_t = (\mathbf{X}, q, h_t)$ pairs the recording with the interaction history $h_t = (o_0, a_1, o_1, \dots, a_{t-1}, o_{t-1})$, where o_0 is the initial observation the environment exposes before any action is taken (the question, and, optionally, the recording itself) and the policy selects an action from

$$\mathcal{A} = \mathcal{C} \cup \{\text{FINAL}(\hat{y})\},$$

either executing code $c \in \mathcal{C}$ in the sandbox or committing a structured answer that terminates the episode. Executing code returns an observation $o_t = \text{exec}(c_t, s_t)$ for $t \geq 1$ appended to the history, and the episode terminates with a single verifiable reward $r(\hat{y}, y) \in [0, 1]$. The Single-pass prediction is the special case $N = 1$ in which $\mathcal{A} = \{\text{FINAL}(\hat{y})\}$ and o_0 already contains the signal, so the policy responds directly from this observation, differing from the recursive case in both the action space and the content of the initial observation.

3.2 The AnomalyXL Benchmark

AnomalyXL is a long-context, multi-channel benchmark for evidence-grounded anomaly localization: every answer requires precise retrieval of a time span, channel, or magnitude (Fig. 2). We generate samples from a stochastic noise process with optional periodic/trend components, z-normalized per channel ($x \leftarrow (x - \mu_x)/\sigma_x$) so injected anomalies keep consistent scale across heterogeneous base signals. Contexts span 16k–131k points and up to 16 channels.

Two formulations. Generated samples are posed in two answer formats. **AnomalyXL-MCQ** asks each question against a fixed label set and is scored by F_1 over six categories: presence, onset before or after a given timestamp, type, magnitude range, affected channel, and lead versus lag (§E.7). **AnomalyXL-Localize** instead asks for a structured JSON summary of every detected anomaly (see Appendix D.1), scored by task-specific continuous metrics in $[0, 1]$ across five tasks:

- **Localize** (1 channel) — detect the presence of an anomaly and return the window, scored by Intersection over Union (IoU) (Appendix E.2).
- **Classify** (1) — return the anomaly type, scored $1\{\hat{k} = k\} \cdot \text{IoU}$, so a wrong type scores zero (Appendix E.3).
- **Magnitude** (1) — report m , with credit decaying linearly in relative error and reaching zero at 50% (Appendix E.4).
- **All channels** (16) — return every anomalous channel and window, scored by set-matched F_1 over events, where a prediction matches when both the channel and an overlapping window are correct (Appendix E.5).
- **Lead-lag** (2) — return the direction and lag, with credit only for the correct direction, then decaying with relative error in the reported offset (Appendix E.6).

Temporal-overlap calculations (see E.1), aggregation procedures, and the task–metric correspondence (Table E.8) are given in Appendix D. Anomalies are injected by applying a localized transformation to a window $x_{s:e}$, scaled by a magnitude m (in normalized channel units) drawn uniformly from $[2, 6]$, or log-uniformly from $[1, 100]$ for the magnitude task. A *level shift* adds a signed offset $\pm m$ over $x_{s:e}$;

a *transient spike* adds a triangular pulse peaking at $\pm m$; a *seasonality* anomaly adds a sinusoid of amplitude m ; a *variance* anomaly adds noise $\epsilon_t \sim \mathcal{N}(0, m^2)$; and a *trend* anomaly adds a ramp from 0 to $\pm m$ that then persists. Localize and all-channels include first-class no-anomaly cases (19.5% and 23% of rows; 7.8% of the test split).

4 Experiments

In the following, we describe our experiments for evaluating TimeRLM. In Section 4.1, we evaluate TimeRLM on both AnomalyXL-MCQ and AnomalyXL-Localize, compare it against TSLM and classical baselines, and describe an experiment for improving performance using reinforcement learning. Section 4.2 describes the evaluation on ARFBench, while Section 4.3 shows how we assess zero-shot performance on an unseen holdout set from the clinical domain.

4.1 Evaluation on AnomalyXL

We evaluate four inference-time configurations on AnomalyXL: single-pass LLMs and TimeRLM, each in both text and image variants. Unless explicitly stated, all are evaluated without any training, using Qwen3.5-4B (Qwen 2026), GPT-4.1-mini (OpenAI et al. 2023) and GPT-5.5 (Singh et al. 2025) as progressively more powerful backbones. We disable sub-agent recursion for RLMs by default unless explicitly mentioned (marked as “depth=1”). The single-pass text variant serializes raw values into the prompt, uniformly subsampling at a fixed stride on long recordings when needed to fit the context window, following prior time-series question-answering protocols (Xie et al. 2026). For the image variant, the single-pass formulation receives a rendered plot of the signal instead of the serialized raw values (see Section 3.1 for details).

Baselines We compare against three purpose-built TSLMs using the same Qwen3.5-4B backbone: ChatTS (Xie et al. 2025) (full resolution text), OpenTSLM-Flamingo (Langer et al. 2026) (compressed latents), and ITFormer (Wang et al. 2025b) (compressed tokens). We instantiate all three architectures with a frozen pretrained Chronos-2 time-series encoder (Ansari et al. 2025) and a LoRA-adapted Qwen3.5-4B LLM backbone (Hu et al. 2022). We further benchmark *Toto-1.0-QA-Experimental*, a Time Series Foundation Model (TSFM)–VLM hybrid introduced with ARFBench (Xie et al. 2026; Cohen et al. 2026), which processes both the time-series and pre-rendered plots. It uses a Qwen3-VL-32B LLM backbone and a frozen Toto time-series encoder (Cohen et al. 2026). We fine-tune separate variants for AnomalyXL-MCQ and AnomalyXL-Localize, allowing each adapter to learn the corresponding answer format. Each variant is trained on $\approx 5,000$ generated examples for 5 epochs, with early stopping based on validation loss. We also evaluate Toto without SFT (denoted ‘No SFT’) using the open-source checkpoint, as it was trained for anomaly localization (Xie et al. 2026). All supervised baselines use the same LoRA configuration, with hyperparameter details provided in Appendix C. We also include a classical baseline with no language model on the same AnomalyXL train/validation splits used for the TSLMs. We pair

standard anomaly-detection features (robust- z exceedance geometry, half-series level and variance contrasts, rolling-variance and spectral change, PELT change points (Killick, Fearnhead, and Eckley 2012), and matrix-profile disorders (Yeh et al. 2016)) with one gradient-boosted head each for AnomalyXL-MCQ and AnomalyXL-Localize (Chen and Guestrin 2016) (Appendix B).

Improving performance via Reinforcement Learning on AnomalyXL. Since detecting and localizing anomalies yields verifiable rewards, we investigate whether reinforcement learning can further improve performance. We post-train TimeRLM with a Qwen3.5-4B RLM backbone using Group Relative Policy Optimization (GRPO) (Shao et al. 2024) on AnomalyXL-Localize, with full-parameter tuning and no supervised warm start. Rather than relying on a fixed training set, we integrate the generator directly into the training loop: every step synthesizes a fresh batch of recordings, so no sample repeats within a run. Formally, for a task with evidence label y , each rollout i commits an answer $\hat{y}_i$ and receives a score $s_i = r(\hat{y}_i, y) \in [0, 1]$, where r is the outcome of the decision process, computed with the same AnomalyXL-Localize parser and per-category metric. We train on the penalized rollout reward

$$R_i = s_i - \lambda_{\text{tok}} s_i \kappa_i - \lambda_{\text{fail}} \mathbf{1}\{\text{forced final}_i\},$$

where $\kappa_i \in [0, 1]$ is the group-normalized token cost of rollout i and $\lambda_{\text{tok}}, \lambda_{\text{fail}}$ weight the two cost penalties. Letting T_i denote the number of tokens generated by rollout i (across all turns of the episode, excluding sandbox observations), we normalize within the group of G rollouts on the same task,

$$\kappa_i = \frac{T_i}{\max_{j \leq G} T_j},$$

so that cost is measured relative to the most verbose rollout of the same task rather than against an absolute budget, leaving exploration on harder tasks unpenalized. For the same reason, the token-cost term applies only to rollouts with nonzero task reward ($s_i > 0$). For each sampled task we draw a group of G rollouts and compute Dr. GRPO advantages from the penalized rewards,

$$A_i = R_i - \frac{1}{G} \sum_{j=1}^G R_j$$

without group-standard-deviation normalization (Liu et al. 2025). Groups with zero reward variance yield $A_i = 0$ for every rollout and are dropped (Yu et al. 2026).

We train for 300 steps with group size $G = 8$, batch size 128, and a 64k-token training sequence length. Rollouts follow the TimeRLM text variant environment with a budget of 15 agent turns, a token-cost coefficient $\lambda_{\text{tok}} = 0.1$, and a flat forced-final penalty $\lambda_{\text{fail}} = 0.25$. We report final checkpoint performances. Anomaly-free cases are withheld for the first 220 steps and reintroduced thereafter, preventing early rollouts from over-using the “no anomaly” answer before the policy has reinforced anomaly-localization behavior.

4.2 Evaluation on ARFBench Tier 2

Beyond AnomalyXL, we evaluate TimeRLM on the Tier 2 tasks of ARFBench (Xie et al. 2026). This includes 306 questions that ask for the properties of an anomaly (identification, start, end, magnitude, and categorization) in multiple-choice form over real telemetry data, comparable to AnomalyXL-MCQ. We instantiate TimeRLM using Qwen3.5-4B zero-shot with and without RL on AnomalyXL-Localize and GPT-5.5 on its text and image variants. As baselines, we use GPT-5.5 in the same text and image variants as detailed in Section 4.1. Additionally, we explore Toto-1.0-Qwen3 32B as TSLM and Toto-1.0-QA-Experimental 32B as TSVLM, one reported from the paper’s results as no checkpoint is provided and the other using the checkpoint provided by the authors (Xie et al. 2026).

4.3 Evaluating generalization to unseen data

To evaluate generalization to unseen tasks, we construct holdout test sets from clinical domains by sampling 250 windows from the Long-Term Atrial Fibrillation (LTAF) ECG dataset (Petruitiu, Sahakian, and Swiryn 2007) and 250 from the Sleep Polysomnography (PSG) corpus (Ghassemi et al. 2018). The LTAF windows contain two channels, each with 7.7k–130k samples, while the Sleep-PSG windows contain four respiratory channels, each with 12k–128k samples. We formulate each window as a “classify-with-evidence” task, following AnomalyXL-Localize, and report macro-F1 for classification and intersection over union (IoU) for localization (Appendix F). Recordings are derived from windows of one to seventeen minutes, each containing at most one annotated anomaly, such as an ectopic beat in an ECG signal or a respiratory arousal in a sleep recording. Windows are selected so that no neighboring annotated anomaly is present, while 20% of the windows are anomaly-free to provide challenging negative examples. The median anomaly occupies 0.5% of an LTAF window and 5.7% of a Sleep-PSG window. As baselines, we evaluate the same TSLMs used for AnomalyXL (Section 4.1). None of the models are trained on these datasets.

4.4 Results

In the following we highlight results and findings based on the experiments introduced in Sections 4.1 to 4.3.

AnomalyXL Table 1 provides an overview of the results on AnomalyXL. On AnomalyXL-MCQ, TimeRLM with a GPT-5.5 backbone with images and GPT-5.5 with text and depth recursion lead on identification and indicator, reaching 0.921 and 0.976, against 0.803 and 0.784 for the best single-pass configuration. Across the remaining tasks spanning presence, start time, categorization, and magnitude, the fine-tuned 32B Toto wins categorization (0.728), and single-pass GPT-5.5 (image) wins on the presence (1.000) task (Table 1). The three TSLMs (ChatTS, OpenTSLM, and ITFormer) all score 0.467 on presence, and achieve 0.144, 0.127 and 0.014 on start time. The Qwen3.5-4B single-pass image baseline reaches 0.897 on start time. The image variant improves over its text counterpart on presence and start time for all three backbones, while text remains

Model		AnomalyXL-MCQ						AnomalyXL-Localize				
		Pres(F1) (40)	ST(F1) (40)	Cat(F1) (42)	Mag(F1) (42)	Id(F1) (42)	Ind(F1) (42)	Loc(IoU) (200)	C+E(cls-IoU) (200)	MMag(err) (200)	LAC(F1) (100)	LL(cls-err) (100)
<i>Classical</i> Features + GBT heads		0.861	0.981	0.643	0.945	0.739	0.521	0.160	0.022	0.685	0.549	0.108
<i>TSLM</i>	ChatTS Qwen3.5-4B	0.467	0.144	0.137	0.118	OOM*	0.175	0.003	0.000	0.092	OOM*	0.350
	OpenTSLM Qwen3.5-4B	0.467	0.127	0.101	0.118	0.010	0.167	0.008	0.000	0.079	0.230	0.350
	ITFormer Qwen3.5-4B	0.467	0.014	0.146	0.183	0.333	0.254	0.004	0.001	0.078	0.230	0.333
	Toto Qwen3-VL 32B (no SFT)	0.804	0.591	0.226	0.362	0.098	0.391	0.026	0.001	0.285	0.220	0.328
	Toto Qwen3-VL 32B	0.812	0.921	0.728	0.944	0.303	0.367	0.323	0.072	0.822	0.224	0.557
<i>Single-pass</i>	GPT-4.1-mini (text)	0.640	0.194	0.215	0.120	0.219	0.167	0.005	0.000	0.154	0.000	0.139
	GPT-5.5 (text)	0.886	0.620	0.420	0.774	0.803	0.476	0.096	0.001	0.506	0.170	0.212
	Qwen3.5-4B (text)	0.729	0.163	0.258	0.476	0.000	0.199	0.035	0.001	0.034	0.000	0.122
	GPT-4.1-mini (image)	0.902	0.685	0.174	0.140	0.143	0.280	0.139	0.009	0.197	0.100	0.191
	GPT-5.5 (image)	1.000	0.929	0.189	0.671	0.520	0.784	0.329	0.043	0.440	0.120	0.753
	Qwen3.5-4B (image)	0.886	0.897	0.191	0.123	0.251	0.668	0.105	0.012	0.051	0.065	0.348
TimeRLM text (ours)	GPT-4.1-mini	0.631	0.594	0.270	0.154	0.374	0.529	0.273	0.075	0.352	0.088	0.291
	GPT-5.5	0.640	0.916	0.567	0.793	0.845	0.952	0.677	0.717	0.496	0.461	0.958
	GPT-5.5 (depth-1)	0.937	0.895	0.585	0.846	0.921	0.976	0.656	0.745	0.554	0.551	0.978
	Qwen3.5-4B	0.687	0.673	0.325	0.445	0.253	0.684	0.205	0.119	0.272	0.003	0.398
	Qwen3.5-4B (RL)	0.739	0.712	0.252	0.505	0.354	0.815	0.538	0.364	0.510	0.244	0.687
TimeRLM image (ours)	GPT-4.1-mini	0.657	0.785	0.240	0.133	0.330	0.553	0.369	0.126	0.389	0.076	0.350
	GPT-5.5	0.765	0.933	0.432	0.801	0.886	0.976	0.682	0.693	0.480	0.537	0.988
	Qwen3.5-4B	0.664	0.793	0.307	0.505	0.381	0.675	0.208	0.109	0.176	0.004	0.345

Table 1: AnomalyXL results. For metrics see Table 9. Number of samples listed below each task. LLMs/RLMs are zero-shot, TSLMs are fine-tuned. **Bold/underline** marks best/second-best. *MCQ*: Pres = presence, ST = start time, Cat = categorization, Mag = magnitude, Id = identification, Ind = indicator. *Localize*: Loc = localize, C+E = classify + evidence, MMag = measure magnitude, LAC = localize all channels, LL = lead/lag + magnitude. * *OOM*: Out of Memory.

better on magnitude (0.476 vs. 0.123 for Qwen3.5-4B). On AnomalyXL-Localize the gap widens on the tasks scored with IoU. TimeRLM (image, GPT-5.5) leads on localization with 0.682 IoU, against 0.329 for the best single-pass variant and 0.323 for fine-tuned 32B Toto. On classify-with-evidence every baseline stays below 0.08 while TimeRLM reaches 0.745. TimeRLM with recursion depth-1 gives small gains on tasks requiring precise segment localization (C+E 0.717 to 0.745, LAC 0.461 to 0.551) using an average of 1.14 sub-RLM calls per task. TimeRLM on a Qwen3.5-4B improves localization from 0.035 to 0.205 IoU compared to the single-pass text variant, and to 0.538 after RL post-training on AnomalyXL, above both the 32B Toto (0.323) and single-pass GPT-5.5 (0.329). Classify-with-evidence rises from 0.001 to 0.364 for the same configuration. On measure magnitude (MMag), the task requiring a single scalar answer, Toto achieves 0.822 and the classical baseline 0.685. The best TimeRLM variant based on GPT-5.5 with image modalities achieves (0.551).

ARFBench On ARFBench Tier 2, TimeRLM leads both metrics: the image variant reaches 61.4% accuracy and 57.9% macro-F1, ahead of single-pass GPT-5.5 (59.5%, 55.8%) and both Toto variants (Table 2). This pattern is consistent with AnomalyXL-MCQ, where TimeRLM on a GPT-5.5 backbone leads among the LLM-based methods, with Toto Qwen3-VL 32B (SFT) close in performance.

		Iden. (38)	Start (56)	End (32)	Magn. (76)	Categ. (104)	Total (306)
Accuracy	<i>n</i>						
	GPT-5.5 text	47.4	37.5	65.6	48.7	58.7	51.6
	GPT-5.5 image	34.2	51.8	71.9	63.2	66.3	59.5
	Toto (TSLM)*	10.5	35.7	34.4	47.4	71.2	47.4
	Toto (TSVLM)†	50.0	23.2	46.9	59.2	64.4	52.0
	<i>TimeRLM-text (Qwen3.5 4B)</i>	31.6	30.4	46.9	36.8	31.7	34.3
	<i>TimeRLM-text (Qwen3.5 4B - RL)</i>	65.8	26.8	40.6	48.7	33.7	40.8
	<i>TimeRLM-text (GPT 5.5)</i>	42.1	50.0	65.6	51.3	60.6	54.6
Macro-F1	<i>TimeRLM-image (GPT 5.5)</i>	60.5	50.0	65.6	64.5	64.4	61.4
	GPT-5.5 text	49.1	36.3	51.8	46.7	53.0	47.8
	GPT-5.5 image	32.4	51.6	56.1	59.4	64.0	55.8
	Toto (TSLM)*	17.5	41.3	23.0	35.9	66.2	43.6
	Toto (TSVLM)†	50.0	18.9	36.2	48.9	56.2	44.7
	<i>TimeRLM-text (Qwen3.5 4B)</i>	44.5	42.7	41.3	46.2	24.0	37.3
	<i>TimeRLM-text (Qwen3.5 4B - RL)</i>	74.5	41.1	42.2	68.3	29.0	48.0
	<i>TimeRLM-text (GPT 5.5)</i>	42.5	49.4	51.6	44.3	53.4	48.9
	<i>TimeRLM-image (GPT 5.5)</i>	58.7	49.4	52.8	60.3	61.9	57.9

* Toto-1.0-Qwen3 32B (TSFM-LLM), numbers taken from the ARFBench paper

† Toto-1.0-QA-Experimental 32B (TSFM-VLM), reproduced (our local rerun)

Table 2: ARFBench Tier 2 results. Accuracy and Macro-F1 as defined in Table 9. **Bold/underline**: best/second.

Generalization set (ECG + sleep). On zero-shot evaluation, TimeRLM leads both sets on classification and localization (Table 3). TSLMs retain some ability to clas-

sify, but no TSLM exceeds an IoU of 0.004. This includes Toto, whose fine-tuned version outperforms some TimeRLM configurations on localization in AnomalyXL-Localize. TimeRLM with recursion spawns an average of 2.6 sub-agents per task, compared to 1.14 in AnomalyXL-Localize, and increases performance overall. The TimeRLM

	Model	LTAF (ECG)		Sleep-PSG	
		F1	IoU	F1	IoU
<i>TSLMs</i>	ChatTS	0.000	0.000	OOM*	OOM*
	OpenTSLM	0.000	0.000	0.083	0.004
	ITFormer	0.331	0.000	0.330	0.000
	Toto (No SFT) [†]	0.000	0.000	0.027	0.000
	Toto (SFT) [‡]	0.000	0.000	0.000	0.000
<i>TimeRLM-text (Ours)</i>	GPT-4.1-mini	0.457	0.025	0.255	0.031
	GPT-5.5	0.724	0.165	0.262	0.147
	GPT-5.5 (depth-1)	0.798	0.271	0.348	0.270
	Qwen3.5-4B	0.357	0.012	0.174	0.011
	Qwen3.5-4B (RL)	0.446	0.016	<u>0.340</u>	0.002
<i>TimeRLM-image (Ours)</i>	GPT-4.1-mini	0.414	0.052	0.201	0.063
	GPT-5.5	<u>0.780</u>	<u>0.192</u>	0.314	<u>0.220</u>
	Qwen3.5-4B	0.278	0.009	0.168	0.004

* Out of memory | [†] Toto-1.0-QA-Experimental 32B | [‡] SFT on AnomalyXL-Localize

Table 3: Zero-shot results. Performance on unseen LTAF and Sleep sets. **Bold/underline:** best/second per column.

variant trained using reinforcement learning on AnomalyXL maintains good performance with F1 rising from 0.36 to 0.45 on LTAF and from 0.17 to 0.34 on Sleep-PSG. However, performance on localization stays near zero (Table 3).

5 Discussion and Conclusion

Our work investigated whether recursive LLMs that can query time-series data through code can improve performance on precise anomaly localization by (1) introducing a TimeRLM formulation, (2) introducing AnomalyXL as a new benchmark, and (3) evaluating performance across AnomalyXL, ARFBench, and an unseen holdout set. Our results support three readings: using RLMs is most advantageous on AnomalyXL-Localize and stems from the fact that the model is able to query the signal multiple times. The RLM approach boosts performance across both text and vision variants. Post-training a small open model on the RLM approach helps to maintain capabilities on zero-shot transfer to real out-of-domain signals.

Multiple choice questions limit retrieval measurement. Classical baselines, i.e., specialized models for anomaly detection, reach the highest weighted average (0.81) on AnomalyXL-MCQ, above every LLM, TSLM, and RLM we evaluated. The same baseline models however, do not outperform TimeRLM under precise scoring, indicating that what they do not recover is the location of an anomaly itself. These results show that MCQ tasks such as presence, start-time ranges, and magnitude bands are often solvable from aggregate signal statistics, allowing systems to score well without precise localization. AnomalyXL-MCQ closely matches related works in anomaly localization

tasks (Cai et al. 2024; Xie et al. 2026). AnomalyXL-MCQ is comparable to ARFBench tier 2 (Xie et al. 2026), and we find that the relative performances of each model, i.e., Toto vs. TimeRLM, are comparable across both benchmarks indicating similarity. These findings suggest that the multiple-choice format commonly utilized in existing TSLM anomaly detection benchmarks over-reports and under-investigates the retrieval abilities of TSLM anomaly localization architectures.

Recursion increases anomaly localization capabilities. On AnomalyXL-MCQ, frontier backbones perform strongly in the single-pass setting: GPT-5.5 image reaches 1.000 on presence and 0.929 on start time. TimeRLM redistributes performance across metrics, trading presence (1.000 to 0.765) for gains in identification (0.520 to 0.886) and categorization (0.189 to 0.432). On AnomalyXL-Localize, however, TimeRLM variants improve on all five tasks (Table 1). The largest gains occur on tasks requiring citing specific segments, including classify-with-evidence (0.043 to 0.693) and localization (0.329 to 0.682). Measured magnitude, which requires a scalar answer, is the exception and remains nearly flat (0.440 to 0.480 for image and 0.506 to 0.496 for text). Rendering the signal as an image helps in the MCQ single-pass setting: for GPT-5.5, it raises start time from 0.620 to 0.929 and localization from 0.096 to 0.329. This advantage does not carry over to TimeRLM. Text and image variants score within 0.03 IoU on localization (0.677 vs. 0.682) and within 0.03 on classify-with-evidence (0.717 vs. 0.693), while the image variant uses 3.1 more agent turns on average. TimeRLM performs best when allowed to spawn sub-agents that examine individual signal segments in depth, reaching 0.745 on classify-with-evidence, 0.551 on localize-all-channels, and 0.978 on lead/lag, while recovering presence to 0.937. These gains align with prior work on agentic search in other modalities (Zhang, Kraska, and Khat-tab 2026; Wang et al. 2024b) and motivate further work on scaling inference-time search for TSLMs and using coding agents for long-context time-series retrieval.

On-policy learning and out-of-distribution transfer. Fine-tuned on AnomalyXL, TSLMs collapse on the unseen LTAF and Sleep-PSG generalization sets, with no model exceeding an IoU of 0.004. ITFormer reaches 0.331 F1 on LTAF and 0.330 on Sleep-PSG. By contrast, reinforcement learning on the AnomalyXL generator improves TimeRLM with Qwen3.5-4B on clinical signal classification, AnomalyXL-MCQ and ARFBench. These results suggest that post-training on synthetic data preserves capabilities on unseen datasets while substantially improving out-of-domain transfer. We hypothesize that reasoning in code provides a signal-agnostic learning environment: the action space remains invariant across signal distributions, allowing the model to learn general procedures for time-series anomaly localization. Related work similarly suggests that on-policy learning can reduce train-test mismatch and preserve prior capabilities better than standard off-policy Supervised Fine-Tuning (SFT) (Shenfeld et al. 2026). For this reason, clean ablations separating code access and SFT-matched post-training remain necessary.

Limitations and Future Work. To our knowledge, AnomalyXL is the first synthetic long-context anomaly benchmark released with its generator, enabling configurable difficulty. Its programmatic construction, however, limits the diversity of anomaly morphologies: injected events are parametric, whereas real anomalies may be ambiguous and context-dependent. AnomalyXL therefore primarily evaluates precise retrieval of numerical deviations rather than the full range of semantically meaningful anomalies. Although our generalization set partly addresses this limitation, future work should study long-context time-series localization on real-world data and beyond the clinical domains considered here. Direct comparisons between Toto and the other TSLMs should also be interpreted cautiously. Toto uses a time-series encoder pretrained on real observability data, is already post-trained for anomaly localization on real telemetry, and employs a 32B VLM, whereas the general-purpose TSLMs share a 4B backbone.

Acknowledgments

We thank Prime Intellect for providing open-source infrastructure for RL environments and recursive language models, compute resources, and technical guidance that supported this work.

References

- Ansari, A. F.; Shchur, O.; Küken, J.; Auer, A.; Han, B.; Mercado, P.; Rangapuram, S. S.; Shen, H.; Stella, L.; Zhang, X.; et al. 2025. Chronos-2: From univariate to universal forecasting. *arXiv Preprints*.
- Boniol, P.; Liu, Q.; Huang, M.; Palpanas, T.; and Paparizos, J. 2024. Dive into Time-Series Anomaly Detection: A Decade Review. *arXiv:2412.20512*.
- Cai, Y.; Choudhry, A.; Goswami, M.; and Dubrawski, A. 2024. TimeSeriesExam: A Time Series Understanding Exam. In *NeurIPS Workshop on Time Series in the Age of Large Models*.
- Chan, S.; Yuan, H.; Tong, C.; Acquah, A.; Schonfeldt, A.; Gershuny, J.; and Doherty, A. 2024. CAPTURE-24: A large dataset of wrist-worn activity tracker data collected in the wild for human activity recognition. *Scientific Data*, 11(1): 1135.
- Chen, M.; Sun, L.; Li, T.; sunhaoze; ZhouYijie; Zhu, C.; Wang, H.; Pan, J. Z.; Zhang, W.; Chen, H.; Yang, F.; Zhou, Z.; and Chen, W. 2026. ReSearch: Learning to Reason with Search for LLMs via Reinforcement Learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Chen, T.; and Guestrin, C. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Cohen, B.; Khwaja, E.; Doubli, Y.; Lemaachi, S.; Lettieri, C.; Masson, C.; Miccinilli, H.; Ramé, E.; Ren, Q.; Ros-tamizadeh, A.; du Terrail, J. O.; Toon, A.-M.; Wang, K.; Xie, S.; Xu, Z.; Zhukova, V.; Asker, D.; Talwalkar, A.; and Abou-Amal, O. 2026. This Time is Different: An Observability Perspective on Time Series Foundation Models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Dingjie, S.; Chen, S.; Chen, G. H.; Yu, F.; Wan, X.; and Wang, B. 2024. MileBench: Benchmarking MLLMs in Long Context. In *First Conference on Language Modeling*.
- Ghassemi, M. M.; Moody, B.; Lehman, L.; Song, C.; Li, Q.; Sun, H.; Westover, M.; and Clifford, G. D. 2018. You Snooze, You Win: the PhysioNet/Computing in Cardiology Challenge 2018. In *2018 Computing in Cardiology Conference (CinC)*, 1–4.
- He, P.; Wen, Z.; Wang, Y.; Wang, Y.; Liu, X.; Huang, J.; Lei, Z.; Gu, Z.; Jin, X.; Yang, J.; et al. 2025. Audiomarathon: A comprehensive benchmark for long-context audio understanding and efficiency in audio llms. *arXiv Preprints*.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- Jaegle, A.; Gimeno, F.; Brock, A.; Zisserman, A.; Vinyals, O.; and Carreira, J. 2021. Perceiver: General Perception with Iterative Attention. In *International Conference on Machine Learning (ICML)*, 4651–4664. PMLR.
- Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S. O.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In *Second Conference on Language Modeling*.
- Kamradt, G. 2023. Needle in a Haystack - Pressure Testing LLMs. GitHub: <https://github.com/gkamradt/LLMTest-NeedleInAHaystack>. Accessed: 2025.
- Kelly, J.; and Knottenbelt, W. 2015. The UK-DALE dataset, domestic appliance-level electricity demand and whole-house demand from five UK homes. *Scientific Data*, 2(1): 150007.
- Killick, R.; Fearnhead, P.; and Eckley, I. A. 2012. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500): 1590–1598.
- Kurатов, Y.; Bulatov, A.; Anokhin, P.; Rodkin, I.; Sorokin, D.; Sorokin, A.; and Burtsev, M. 2024. Babilong: Testing the limits of llms with long context reasoning-in-a-haystack. *Advances in Neural Information Processing Systems (NeurIPS)*, 37: 106519–106554.
- Langer, P.; Kaar, T.; Rosenblattl, M.; Xu, M. A.; Chow, W.; Maritsch, M.; Jakob, R.; Wang, N.; Liu, J.; Verma, A.; Han, B.; Kim, D. S.; Chubb, H.; Ceresnak, S. R.; Zahedi-vash, A.; Sandhu, A. T.; Rodriguez, F.; McDuff, D.; Fleisch, E.; Aalami, O. O.; Barata, F.; and Schmiedmayer, P. 2026. OpenTSLM: Time-Series Language Models for Reasoning over Multivariate Medical Text- and Time-Series Data. In *Forty-third International Conference on Machine Learning*.
- Liu, N. F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; and Liang, P. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12: 157–173.

- Liu, Z.; Chen, C.; Li, W.; Qi, P.; Pang, T.; Du, C.; Lee, W. S.; and Lin, M. 2025. Understanding R1-Zero-Like Training: A Critical Perspective. In *Second Conference on Language Modeling*.
- Messica, S.; Zhang, J.; Li, K.; Tsiligkaridis, T.; and Zitnik, M. 2026. Adaptive Time Series Reasoning via Segment Selection. In *Forty-third International Conference on Machine Learning*.
- Oh, J.; Lee, G.; Bae, S.; Kwon, J.-m.; and Choi, E. 2023. Ecg-qa: A comprehensive question answering dataset combined with electrocardiogram. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 66277–66288.
- OpenAI; et al. 2023. GPT-4 Technical Report. *arXiv Preprints*.
- Petrutiu, S.; Sahakian, A. V.; and Swiryn, S. 2007. Abrupt Changes in Fibrillatory Wave Characteristics at the Termination of Paroxysmal Atrial Fibrillation in Humans. *Europace*, 9(7): 466–470.
- Qwen, T. 2026. Qwen3.5-Omni Technical Report. *arXiv:2604.15804*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Zhang, M.; Li, Y.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.
- Shenfeld, I.; Damani, M.; Hübotter, J.; and Agrawal, P. 2026. Self-Distillation Enables Continual Learning. In *Forty-third International Conference on Machine Learning*.
- Singh, A.; et al. 2025. Openai gpt-5 system card. *arXiv Preprints*.
- Su, J.; Jiang, C.; Jin, X.; Qiao, Y.; Xiao, T.; Ma, H.; Wei, R.; Jing, Z.; Xu, J.; and Lin, J. 2024. Large Language Models for Forecasting and Anomaly Detection: A Systematic Literature Review. *ArXiv*, abs/2402.10350.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wang, C.; Qi, Q.; Wang, J.; Sun, H.; Zhuang, Z.; Wu, J.; Zhang, L.; and Liao, J. 2025a. Chattime: A unified multimodal time series foundation model bridging numerical and textual data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 12694–12702.
- Wang, W.; Zhang, S.; Ren, Y.; Duan, Y.; Li, T.; Liu, S.; Hu, M.; Chen, Z.; Zhang, K.; Lu, L.; Zhu, X.; Luo, P.; Qiao, Y.; Dai, J.; Shao, W.; and Wang, W. 2024a. Needle In A Multimodal Haystack. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Wang, X.; Zhang, Y.; Zohar, O.; and Yeung-Levy, S. 2024b. VideoAgent: Long-Form Video Understanding with Large Language Model as Agent. In *ECCV (80)*, 58–76.
- Wang, Y.; Lei, P.; Song, J.; Hao, Y.; Chen, T.; Zhang, Y.; JIA, L.; Li, Y.; and zhongyu wei. 2025b. ITFormer: Bridging Time Series and Natural Language for Multi-Modal QA with Large-Scale Multitask Dataset. In *Forty-second International Conference on Machine Learning*.
- Xie, S.; Cohen, B.; Goswami, M.; Shen, J.; Khwaja, E.; Liu, C.; Asker, D.; Abou-Amal, O.; and Talwalkar, A. 2026. ARFBench: Benchmarking Multimodal Time Series Reasoning for Software Incident Response. In *1st ICLR Workshop on Time Series in the Age of Large Models*.
- Xie, Z.; Li, Z.; He, X.; Xu, L.; Wen, X.; Zhang, T.; Chen, J.; Shi, R.; and Pei, D. 2025. ChatTS: Aligning Time Series with LLMs via Synthetic Data for Enhanced Understanding and Reasoning. *Proceedings of the VLDB Endowment*, 18(8): 2385–2398.
- Xu, M. A.; Haresamudram, H.; Liu, C. W.; Langer, P.; Pradeepkumar, J.; Mao, W.; Ferns, S. J.; Verma, A.; Sun, J.; Schmiedmayer, P.; Liu, X.; McDuff, D.; Fox, E. B.; and Rehg, J. M. 2026. How Well Do Multimodal Models Reason on ECG Signals? *arXiv:2603.00312*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.
- Yèche, H.; Kuznetsova, R.; Zimmermann, M.; Hüser, M.; Lyu, X.; Faltys, M.; and Rätsch, G. 2021. HiRID-ICU-Benchmark – A Comprehensive Machine Learning Benchmark on High-resolution ICU Data. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Yeh, C.-C. M.; Zhu, Y.; Ulanova, L.; Begum, N.; Ding, Y.; Dau, H. A.; Silva, D. F.; Mueen, A.; and Keogh, E. 2016. Matrix Profile I: All Pairs Similarity Joins for Time Series: A Unifying View That Includes Motifs, Discords and Shapelets. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*, 1317–1322.
- Yu, Q.; Zhang, Z.; Zhu, R.; and Yufeng Yuan, e. a. 2026. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Yuan, H.; Chan, S.; Creagh, A. P.; Tong, C.; Acquah, A.; Clifton, D. A.; and Doherty, A. 2024. Self-supervised learning for human activity recognition using 700,000 person-days of wearable data. *NPJ Digital Medicine*, 7(1): 91.
- Zhang, A. L.; Kraska, T.; and Khattab, O. 2026. Recursive Language Models. *arXiv:2512.24601*.
- Zhao, H.; Zhang, X.; Wei, J.; Xu, Y.; He, Y.; Sun, S.; and You, C. 2025. TimeSeriesScientist: A General-Purpose AI Agent for Time Series Analysis. *arXiv:2510.01538*.
- Zumarraga, N.; Kaar, T.; Wang, N.; Tennien, W.; Hasanli, A.; Rosenblattl, M.; Wu, F.; Riehl, K.; Xu, M. A.; Kreft, M.; O’Sullivan, K.; Fleisch, E.; Schmiedmayer, P.; Jakob, R.; and Langer, P. 2026. TS-Haystack: A Multi-Task Retrieval Benchmark for Long-Context Time-Series Reasoning. *arXiv:2602.14200*.

A TimeRLM Details

This section further discusses the per task and context length performance of the best performing RLM configuration in AnomalyXL-Localize, GPT-5.5. We additionally document the exact agent loop underlying TimeRLM during training and evaluation, Prime Intellect’s v1 rlm-harness protocol. The harness is a persistent IPython agent run as a subprocess per rollout, instantiating the decision process of the Methods section: a persistent code sandbox, a terminal structured answer, and a single verifiable reward.

Results by category and context length. The per-category averages in Table 1 show that TimeRLM’s precise gains concentrate on evidence-pointing tasks, but they collapse away the length axis. Fig. 3 breaks GPT-5.5’s precise score down by category *and* context length for all four inference-time configurations, and two patterns emerge that the averaged column hides. First, classify-with-evidence is at or near zero for both single-pass variants at *every* length (text: ≤ 0.003 ; image: $0.11 \rightarrow 0.00$ as L grows), while the RLM holds $0.62\text{--}0.76$ at every length regardless of modality. The collapse is uniform across the length range, not driven by the longest contexts alone. Second, single-pass image localization degrades steadily with length, from 0.48 at $L = 16\text{k}$ to 0.16 at $L = 131\text{k}$, while the RLM’s localize score stays within a narrow $0.61\text{--}0.79$ band over the same range. With the exception for Measure Magnitude (a scalar answer not requiring precise retrieval scored with IoU) and the Localize task for single pass text, results indicate the relative single-pass/RLM gap widens with context length and degrades further in absolute terms from $L = 32\text{k}$ to $L = 131\text{k}$, consistent with scanning 16 channels by eye or by serialized text becoming harder, not easier, as the series lengthens.

Fig. 4 aggregates the three categories swept across all four lengths (localize, classify-with-evidence, measure-magnitude) into a single accuracy-versus-length curve per configuration. Averaged this way, single-pass image score drops by roughly 46% from the shortest to the longest context ($0.35 \rightarrow 0.19$), while single-pass text stays flat but low throughout ($0.18\text{--}0.23$); both RLM curves remain within a tight $0.56\text{--}0.69$ band across the same length range, well above either single-pass curve at every length and with no comparable widening or narrowing trend.

Prompts. The system prompt states a generic context, with task-specific instruction and answer options carried in the per-task prompt:

System prompt

You are a specialized time-series agent that uses code to solve tasks. You solve tasks by breaking down problems into sub-tasks, writing and executing code, observing results, and iterating one step at a time. When you are done, stop calling tools and state your final answer.

Precise prompt

This is a critical application: when relevant, precisely locate the anomaly’s start and end using statistical thresholds --- be precise... Analyze it with numpy and scipy in the IPython tool. When you are done, stop calling the tool and state your final answer as a single JSON object matching the question’s schema exactly (e.g. `{"start": 12, "end": 87}`), with no prose and no markdown fences. Format failures score zero.

Turn budget. During RL, the episode is capped at 15 turns to maintain manageable memory budgets and speed during training; on the final turn, tools are withdrawn and a “*FINAL TURN: your turn budget is exhausted and tools are disabled. State your final answer NOW. . .*” notice is injected, including a -0.25 exhausted-budget penalty used at training time. The episode otherwise ends as soon as the model stops calling tools and states its final answer directly.

Sandbox and packages. The agent executes in an isolated Python sandbox backed by a real Jupyter kernel, pre-seeded with `numpy` and `scipy`. Tool output returned to the model is truncated at 8192 characters per turn (RLM_MAX_OUTPUT). Recursive `sub_rlm()` calls are implemented but disabled (depth 0) throughout, as stated in the Methods section. The answer is extracted by locating a balanced-brace JSON object containing the schema’s required keys, tolerating surrounding prose or markdown fences; a response from which no such object can be recovered scores zero.

B Classical Baseline Details

The classical baseline pairs a deterministic, untrained feature extractor with small supervised task heads. No language model is involved at any stage.

Feature extraction. Series are decimated to at most 16,384 points by stride subsampling, and index-valued predictions are mapped back through the stride. 25 features are computed per channel:

- **Robust- z statistics.** Global median/MAD z -scores: $\max |z|$ and its position, the fraction of points beyond 4σ and 8σ , exceedance-run geometry at $|z| > 4$ (first and last exceedance, longest-run start, end and length, run count), and peak width at $|z| > 6$.
- **Half-series contrasts.** Mean shift between halves (level shift), log variance ratio (variance change), and global linear slope (trend).
- **Rolling-variance localization.** Maximum and argmax of the robust- z of a rolling standard deviation with window $L/64$, capturing variance-change onset.
- **Spectral change.** Dominant-frequency power ratio and frequency shift between halves, via the real FFT, capturing seasonality change.

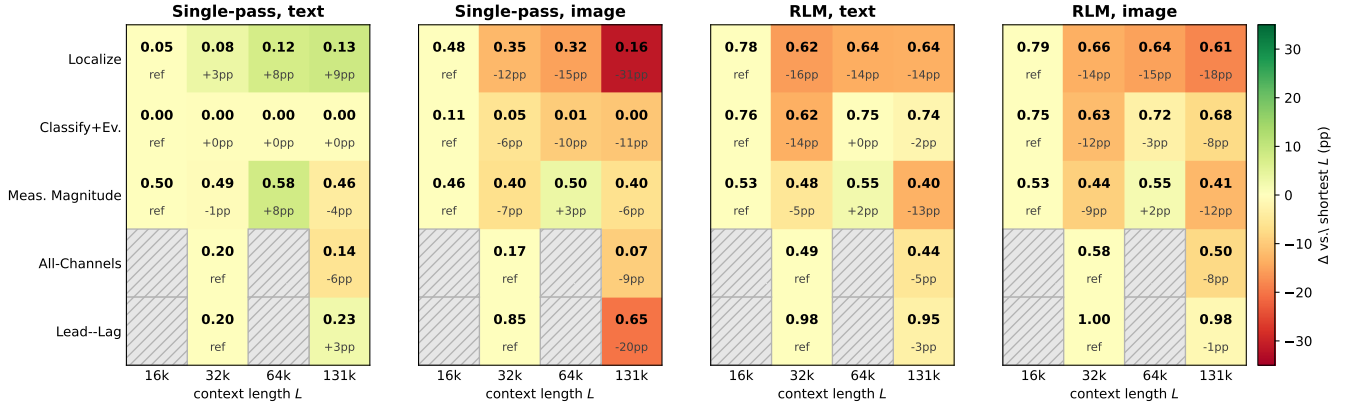


Figure 3: GPT-5.5 precise reward by category and context length, for all four inference-time configurations (verified against Table 1 to three decimals). Each cell shows the mean reward (bold) and its change in percentage points relative to that category’s shortest sampled length; color encodes this delta (green = improves with length, red = degrades).

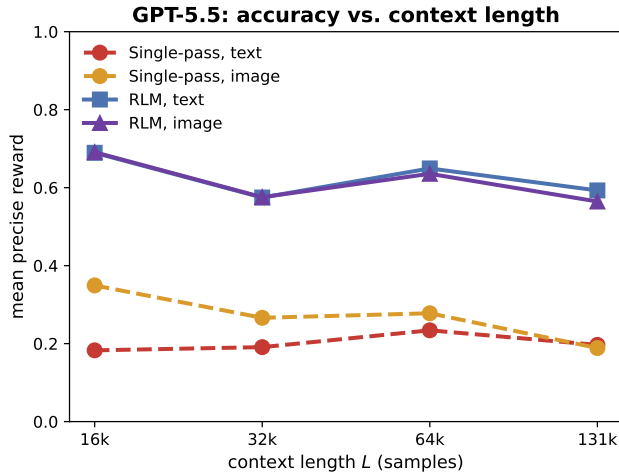


Figure 4: GPT-5.5 accuracy versus context length, averaged over the three length-swept precise categories (localize, classify-with-evidence, measure-magnitude) for each inference-time configuration.

- **Change points.** PELT with an ℓ_2 cost on a 2,048-point grid: breakpoint count, first breakpoint, and the largest inter-segment mean jump with its location.
- **Matrix profile.** Maximum discord value and its location, on a 4,096-point grid with window $L/64$.
- **Series length.** $\log_2$ of the raw, un-decimated recording length.

Each question is then described by 56 values: the worst channel’s full feature vector (25), the channel-mean of every feature (25), the channel count (1), and five cross-correlation features between the two groups—best-lag correlation, lag as a fraction of length, zero-lag correlation, and each group’s own $\max |z|$ —always present but zero-filled outside the lead-lag and indicator tasks ($25 + 25 + 1 + 5 = 56$).

Task heads. Heads are gradient-boosted trees, one per question category (Table 4). They are trained on *exactly* the data used by the TSLM baselines—the AnomalyXL train and validation splits—with an independent set of heads fitted per regime, mirroring the per-regime TSLM and Toto fine-tunes; only the untrained, deterministic feature extractor is shared between them.

Category	Head	Target
<i>Coarse</i>		
Presence	classifier	Yes / No
Start time	classifier	start bin
Categorization	classifier	anomaly kind
Magnitude	classifier	σ band
Identification	per-channel binary	channel in gold set
Indicator	classifier (paired)	5 relations
<i>Precise</i>		
Localize	presence clf. + span regr.	start, end
Classify + ev.	kind clf. + span regr.	kind, start, end
Measure magn.	regressor on $\log(1 + \sigma)$	magnitude_s
Localize all ch.	per-channel clf.	anomaly list
Lead-lag	direction clf. + lag regr.	direction, lag

Table 4: One head per AnomalyXL category. Span regressors predict start and end as fractions of the recording length; intervals for localize-all-channels are read off the longest positive run.

Decoding and evaluation. Head predictions are mapped deterministically into the benchmark’s answer space: coarse heads emit one of the row’s option strings, and precise heads emit the structured JSON objects the benchmark defines. Evaluation is a single pass over the same held-out test sets and the same scorers used for every other row in Table 1.

Because the feature design encodes the task knowledge and the heads learn only thresholds and combinations, this baseline receives, in effect, the information the language-model systems are given in their prompts—the question and

its category definition—but supplied to it by construction rather than read from text.

C TSLM Fine-Tuning Configurations

In the following, we present details on the training configurations of all TSLM baseline results displayed in the paper. All finetuned TSLM baselines share the same training configuration detailed below unless explicitly differentiated, and are separately finetuned on AnomalyXL-MCQ and AnomalyXL-Localize to allow each model variant to learn the answer schema required by each separate task, yielding a total of 8 finetuning runs. An overview of the finetuned model variants is presented in Table 5.

ChatTS (Xie et al. 2025), OpenTSLM-Flamingo (Langer et al. 2026) and ITFormer (Wang et al. 2025b) use a frozen Chronos-2 time-series encoder (Ansari et al. 2025) and a Qwen3.5-4B (Qwen 2026) backbone adapted with LoRA (Hu et al. 2022). Toto-1.0-QA-Experimental (Xie et al. 2026) uses a pretrained Toto encoder (Cohen et al. 2026) attached to a Qwen3-VL-32B backbone, taken directly from the supplied ARFBench paper checkpoint in its base version, and further finetuned when mentioned, under the same conditions of all other TSLMs.

Hyperparameter configurations. All supervised finetunes use batch size 1, 5 epochs, an AdamW optimizer with learning rate $1e^{-4}$, weight decay 0.01, gradient clipping at norm 1.0, and a 0.03 warmup ratio. Early stopping monitors validation loss with patience 3. Validation is run after each epoch on 500 samples from each AnomalyXL-MCQ and AnomalyXL-Localize validation set, scored with the same AnomalyXL scoring rules used in the test set reported Table 1. The checkpoint achieving highest validation primary metric is selected as the evaluated baseline on the test set

LoRA adaptation. All fine-tuned language backbones are adapted with LoRA rather than full-model finetuning, set with rank 16, $\alpha = 32$, dropout 0.05, no bias terms, and attention projection targets q_{proj} and v_{proj} while the time-series encoder remains frozen. For ChatTS, OpenTSLM-Flamingo, and ITFormer, LoRA is applied to the shared Qwen3.5-4B backbone and the architecture-specific time-series projection or fusion parameters are trainable. Toto follows the same optimization hyperparameters and LoRA settings, but applies them to its Qwen3-VL-32B text decoder. Its Toto encoder and projection modules remain frozen.

Data and scoring. The precise regime is trained on approximately 5,000 generated AnomalyXL examples using the same generator configuration as the evaluation set. Coarse models are finetuned to emit one of the multiple-choice options, and precise models to emit a structured JSON answer required by the task. The final reported numbers are computed with the benchmark’s standard scorers: multiple-choice question accuracy for coarse tasks and the continuous evidence-grounded primary score for precise tasks.

D AnomalyXL Construction Details

AnomalyXL signals are used both for precise, open-ended questions answered in strict JSON, and for coarse multiple-choice questions scored by choice accuracy. Table 6 gives the answer schema, metric, and option set for each of the five tasks.

D.1 Templated JSON response examples

Fig. 5 shows six sampled rows from the released AnomalyXL-Localize shard at full resolution (thin gray), with the gold window shaded and, except for the no-anomaly case, a zoomed inset. Panel 1 (Localize, $L = 131,072$) asks whether an anomaly is present and where; gold `{"present": true, "start": 103194, "end": 103352}`. Panel 2 (Classify-with-Evidence, $L = 65,536$) asks for the anomaly’s kind and window; gold `{"kind": "Change in Trend", "start": 55469, "end": 55642}`. Panel 3 (Measure Magnitude, $L = 32,768$) asks for the peak deviation in σ ; gold `{"magnitude-sigma": 13.62}`. Panel 4 (All-Channels, $C=16$, $L = 131,072$) asks for every anomalous channel and its window; gold anomalies fall on `ch_02`, `ch_06`, and `ch_12` (widths 78–233 samples), spread across the full series. Panel 5 (Lead-Lag, $L = 131,072$) asks whether A leads or lags B and by how many samples; gold `{"direction": "lead", "lag-samples": 2051}`. Panel 6 (Localize, no-anomaly case, $L = 16,384$) is one of the first-class negatives; gold `{"present": false}`. The exact question text for Classify-with-Evidence, from the released dataset, reads:

Classify-with-Evidence prompt (verbatim, $L = 65,536$)

```
Identify the anomaly in the time series
and its window. Respond with exactly one
JSON object on a single line, no other
text: {"kind": <label>, "start": <int>,
"end": <int>}. kind must be one of:
"Level Shift", "Transient Spike", "Change
in Seasonality", "Change in Variance",
"Change in Trend". start is inclusive,
end is exclusive. Each kind is defined
as follows: a Level Shift is a sustained
step in mean... (Here L = 65536.)
```

D.2 Scene synthesis.

Every sample starts from a *scene*: C independent channels, each built as one base noise process (white noise, a rescaled random-walk “red” noise, or an AR(1) process with $\phi = 0.7$, drawn with equal probability) plus an optional periodic component (probability 0.4; sine, square, or sawtooth with period in $[16, 256]$ samples) and an optional trend component (probability 0.3; linear, exponential, or logarithmic drift, or a constant offset). Each channel is then z-normalized to unit std, so an anomaly expressed in σ -units has a comparable visual scale regardless of which base processes were drawn. Channels are anonymized (`ch_00`, `ch_01`, ..., or `A/B_-`).

Model row	Signal interface	Fine-tuned regimes
ChatTS Qwen3.5-4B	full-resolution patch adapter	coarse, precise
Flamingo Qwen3.5-4B	Perceiver-Resampler fixed-latent compressor	coarse, precise
ITFormer Qwen3.5-4B	fixed summary-token aggregation	coarse, precise
Toto Qwen3-VL 32B	frozen Toto encoder with Qwen3-VL	coarse, precise

Table 5: Supervised baseline fine-tuning runs. The model names match the rows in Table 1. Each supervised baseline is trained once for the coarse answer format and once for the precise answer format.

Task	Precise answer (JSON)	Metric	Coarse choices (MCQ)
Localize	<code>{"present": true, "start": 48210, "end": 48533}</code>	IoU	Presence: Yes / No; Start Bin: Before / Early / Medium / Late
Classify	<code>{"kind": "Trend", "start": 71040, "end": 73219}</code>	$\text{kind} \times \text{IoU}$	Kind: Spike / Shift / Trend / Variance / Seasonal
Magnitude	<code>{"magnitude_sigma": 12.4}</code>	rel. error	Band: 1–3 / 3–10 / 10–30 / 30–100 σ
All channels	<code>{"anomalies": [{"channel": 4, "start": 9000, "end": 9220}, ...]}</code>	set-matched F1	Count: None / 1 / 2 / 3 of the named channels
Lead-lag	<code>{"direction": "lead", "lag_samples": 512}</code>	dir.+lag	Indicator: Leading / Lagging / Correlated / Uncorrelated

Table 6: The five AnomalyXL tasks, precise versus coarse. Precise answers are strict JSON, scored by the listed continuous metric in $[0, 1]$; coarse answers are scored by choice accuracy. Magnitude’s relative error is converted to a score via $\max(0, 1 - \text{rel_err}/0.5)$, a linear ramp from full credit at an exact match to zero at 50% relative error. The localize no-anomaly case is answered with `{"present": false}`. See Fig. 2 for a schematic and Fig. 5 for worked examples drawn from the released data.

prefixed for the paired lead-lag task) so the model cannot exploit channel-name priors.

D.3 Anomaly injection.

An anomaly is one of five transforms applied to a contiguous window $[t_{\text{start}}, t_{\text{end}})$ of a single target channel, *after* z-normalization. A *level shift* is a signed, sustained step of magnitude $\times \sigma$. A *transient spike* is a triangular pulse peaking at magnitude $\times \sigma$ at the window’s center, with zero at its edges. A *change in seasonality* is a new sinusoid of period in $[4, \text{width}/2]$ samples, short enough that at least two cycles fit inside the window, and added over the window. A *change in variance* is extra Gaussian noise of std magnitude $\times \sigma$ added sample-by-sample over the window, with a width floor raised to 64 samples so the change is a sustained regime rather than indistinguishable from a couple of spikes. On the paired lead-lag task, this minimum is set to 11 samples. Finally, a *change in trend* is a piecewise-linear ramp that reaches $\pm \text{magnitude} \times \sigma$ by the window’s end and then carries its terminal level forward, so it reads as a lasting slope change rather than a transient pulse. Magnitude is drawn uniformly in $[2, 6] \sigma$ for localize, classify, all-channels, and lead-lag, and log-uniformly in $[1, 100] \sigma$ for measure-magnitude, so that task alone stresses both near-floor and extreme deviations. Window width is an *absolute* sample count in $[8, 256]$ regardless of L — not a fraction of the series — which is the design choice that makes long context hard: at $L = 131,072$ an anomaly can occupy as little as 0.006% of the series, indistinguishable from noise in a full-resolution plot unless the reader zooms in, exactly as Fig. 5 shows.

D.4 Channel topologies and sampling grid.

Localize, classify-with-evidence, and measure-magnitude are single-channel ($C = 1$) tasks swept over all four context lengths, $L \in \{16384, 32768, 65536, 131072\}$. Localize-all-channels fixes $C = 16$ and injects $K \sim \mathcal{U}\{0, \dots, 3\}$ anomalies on K distinct channels with non-overlapping windows (channels outside the injected set carry no anomaly), swept over $L \in \{32768, 131072\}$. Lead-lag-with-magnitude splits $C = 2$ channels into single-channel sub-scenes A and B and samples a temporal relation — *lead* (A’s anomaly precedes B’s), *lag* (A follows B), or *independent* (disjoint, unrelated windows) — each with prior $1/3$; for lead/lag the offset is uniform in $[1, L/4]$ samples, so it never exceeds a quarter of the series. Every $(L, \text{category})$ cell draws 50 rows from a seed derived deterministically, so extending the grid later does not reshuffle existing rows. Table 7 summarizes the resulting 800-row precise shard. Fig. 6 plots the classify-with-evidence anomaly-kind distribution.

D.5 Coarse projection.

Rather than synthesizing a second, independent set of scenes, every coarse row is *projected* from a sampled precise row over the identical signal, with the source row’s channels, injections, and coupling copied, and only the question, options, and gold label rewritten into the corresponding MCQ template (Table 8). This is what lets the paired precise/coarse design isolate the scoring axis, as the two regimes never differ in the underlying recording. The resulting shard has 248 rows.

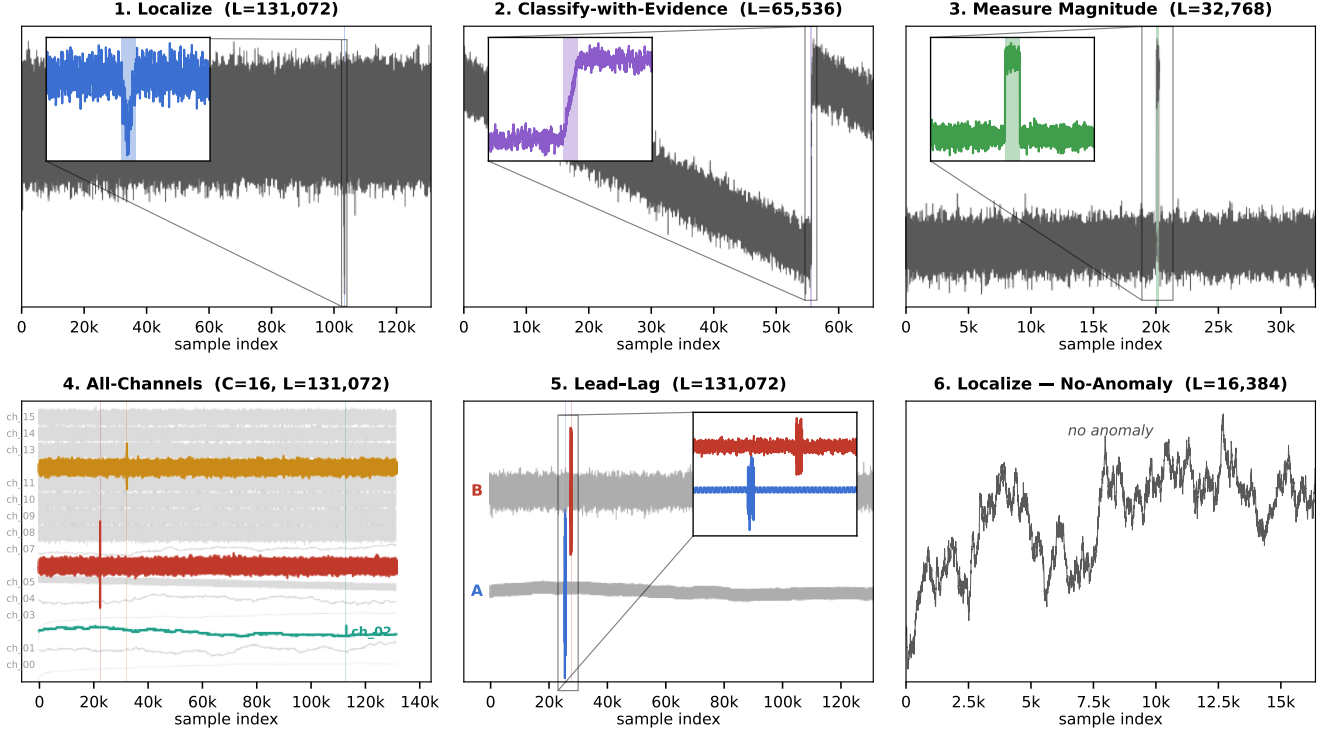


Figure 5: Six worked AnomalyXL-Localize examples, sampled from the released dataset. Full-resolution signal in gray with the gold window shaded; zoomed insets show the injected transform up close. Questions and gold answers are given in the text above.

Category	C	L (samples)	Rows	Composition
Localize	1	16k–131k	200	161 present / 39 absent
Classify-w-Evidence	1	16k–131k	200	Variance 51, Spike 43, Seasonality 37, Trend 37, Shift 32
Measure Magnitude	1	16k–131k	200	$\sigma \in [1.0, 99.8]$, median 12.6
All-Channels	16	32k, 131k	100	$K=0:23, 1:27, 2:25, 3:25$
Lead-Lag	2	32k, 131k	100	lead 33 / lag 32 / independent 35
Total (precise)		16k–131k	800	

Table 7: AnomalyXL-Localize composition (released AnomalyXL-Localize shard, seed 42). Window widths are 8–256 samples throughout, independent of L ; magnitude is in channel- σ units.

E AnomalyXL scoring and metrics

Each precise row is scored by a deterministic function of the model’s final message that returns a value in $[0, 1]$. The same value is used as the evaluation score and, during RL, as the reward.

E.1 Temporal overlap.

Windows are defined as half-open index intervals $w = [s, e)$. For a predicted window $\hat{w} = [\hat{s}, \hat{e})$ and a gold window $w = [s, e)$,

$$\text{IoU}(\hat{w}, w) = \frac{I}{(\hat{e} - \hat{s}) + (e - s) - I}, \quad (1)$$

where $I = \max(0, \min(\hat{e}, e) - \max(\hat{s}, s))$ is the length of the overlap. We set $\text{IoU} = 0$ if the two intervals are disjoint

or if either is degenerate ($\hat{s} \geq \hat{e}$ or $s \geq e$). Every window-based task below uses this definition.

E.2 Localize (r_{loc}).

Let $p, \hat{p} \in \{0, 1\}$ be the gold and predicted presence flags.

$$r_{\text{loc}} = \begin{cases} 1, & p = \hat{p} = 0, \\ \text{IoU}(\hat{w}, w), & p = \hat{p} = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

The second case requires a well-formed predicted window: a `present`: true answer that omits start or end scores 0. A presence mismatch in either direction scores 0.

Precise source	MCQ template(s)	Rows	Option set
Localize	Presence + Start-Time	40×2	Yes/No; 4 quartile bins + No Anomaly
Classify-w-Evidence	Categorization	42	4 of 5 kind labels (1 dropped at random) + No Anomaly
Measure Magnitude	Magnitude	42	4 σ -bins + No Anomaly
All-Channels	Identification	42	2 singletons + 1 pair + triplet + None, over 3 named channels
Lead-Lag	Indicator	42	5 fixed options (Leading / Lagging / Correlated / Uncorrelated / No anomaly)
Total (coarse)		248	

Table 8: Coarse projection rules (AnomalyXL-MCQ, projected from AnomalyXL-Localize with seed 42). Localize yields two coarse rows per sampled precise row (presence and start-time); every other category yields one.

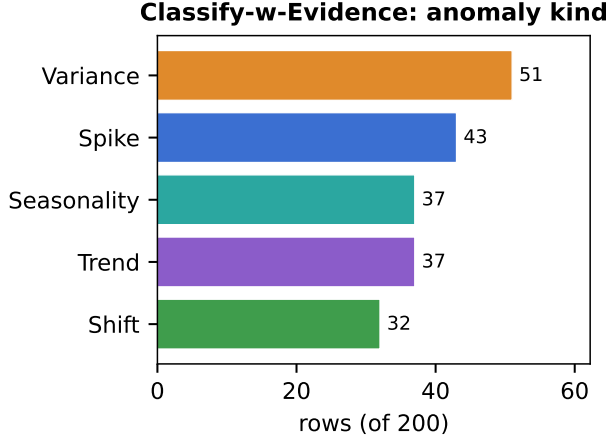


Figure 6: AnomalyXL-Localize dataset composition. Anomaly-kind distribution within classify-with-evidence.

E.3 Classify with evidence (r_{cls}).

The score is the product of a label term and a localization term,

$$r_{\text{cls}} = \mathbf{1}[\hat{k} = k] \cdot \text{IoU}(\hat{w}, w), \quad (3)$$

so a correct label earns credit only when it is paired with a window and the label acts as a scoring gate for the segment localization to score at all. The label $k_{\emptyset}$ = “No anomaly” carries no window and set $r_{\text{cls}} = \mathbf{1}[\hat{k} = k = k_{\emptyset}]$. Labels are compared by exact string equality against the five options given in the prompt.

E.4 Measure magnitude (r_{mag}).

For gold peak deviation $m > 0$ and prediction $\hat{m}$, let $\rho = |\hat{m} - m|/m$. The score is a linear ramp,

$$r_{\text{mag}} = \max\left(0, 1 - \frac{\rho}{\tau}\right), \quad \tau = 0.5, \quad (4)$$

which is 1 at an exact match and 0 at 50% relative error. A non-numeric or absent `magnitude_sigma` sets $\rho = 1$ and hence $r_{\text{mag}} = 0$. Because ρ is a relative error and gold magnitudes are drawn log-uniformly in $[1, 100]\sigma$, errors at the low and high ends of that range are weighted equally.

E.5 All channels (r_{ac}).

This is the only task whose answer is a *list* of events rather than a single window, so the score compares two sets. Each event is a (channel, window) pair: the gold set G has n_g of them and the predicted set P has n_p .

We first decide which predictions found which gold events. A prediction counts as finding a gold event if it names the same channel and its window has $\text{IoU} > \theta$ with the gold window, with $\theta = 0.3$. Pairs are formed greedily, highest IoU first, and each gold event and each prediction is used at most once. Every pair formed this way is a true positive. Predictions left unpaired are false positives, and gold events left unpaired are false negatives. The score is the F_1 of those three counts,

$$r_{\text{ac}} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}, \quad (5)$$

so a model is penalised both for missing events and for inventing them. On a row that contains no anomalies at all, $r_{\text{ac}} = 1$ if the model returns an empty list and 0 otherwise.

E.6 Lead-lag (r_{ll}).

Let $d, \hat{d} \in \{\text{lead}, \text{lag}, \text{independent}\}$ and let $\ell, \hat{\ell}$ be the gold and predicted offsets in samples. The offset earns credit only if the relation is correct:

$$r_{\text{ll}} = \mathbf{1}[\hat{d} = d] \cdot \begin{cases} 1, & d = \text{independent}, \\ \max\left(0, 1 - \frac{\delta}{\tau_{\ell}}\right), & \text{otherwise,} \end{cases} \quad (6)$$

where $\delta = |\hat{\ell} - \ell|/L$ and $\tau_{\ell} = 0.05$. The independent case has no ground-truth offset, so it reduces to three-way classification. For the directional cases the tolerance is a fraction of L , which keeps the task equally hard at every context length given that gold offsets are drawn uniformly in $[1, L/4]$.

E.7 MCQ tasks.

Coarse (MCQ) rows are scored by exact string match of the extracted answer against the gold option text, giving $r \in \{0, 1\}$. Extraction accepts the raw final message, a JSON object of the form `{"answer": "..."}` , or a trailing `answer:` line, in each case after stripping any reasoning block. The paired design means the same recording is scored by a continuous, evidence-grounded metric in the precise regime and by a $\{0, 1\}$ choice indicator in the coarse one, with nothing else differing between the two.

Task	Required output, and quantity parsed from it	Score in $[0, 1]$	Defined in
ANOMALYXL-LOCALIZE — the first schema-valid JSON object in the final message is parsed; a row scores 0 if none can be recovered			
Localize	$\{\text{"present": bool, "start": int, "end": int}\} \rightarrow$ presence flag $\hat{p}$, and window $\hat{w} = [\hat{s}, \hat{e})$ when $\hat{p} = 1$	1 if $p = \hat{p} = 0$; $\text{IoU}(\hat{w}, w)$ if $p = \hat{p} = 1$; 0 on presence mismatch or missing window	Section E.2, Eq. (2)
Classify	$\{\text{"kind": str, "start": int, "end": int}\} \rightarrow$ label $\hat{k}$, matched by exact string equality against the five prompt options, and window $\hat{w}$	$1[\hat{k} = k] \cdot \text{IoU}(\hat{w}, w)$; the label gates the window, so a wrong kind scores 0 however well localized	Section E.3, Eq. (3)
Magnitude	$\{\text{"magnitude_sigma": float}\} \rightarrow$ estimate $\hat{m}$, reduced to relative error $\rho = \hat{m} - m /m$; absent or non-numeric sets $\rho = 1$	$\max(0, 1 - \rho/\tau)$ with $\tau = 0.5$: a linear ramp from full credit at an exact match to zero at 50% relative error	Section E.4, Eq. (4)
All channels	$\{\text{"anomalies": [{"channel": int, "start": int, "end": int}, \dots]}\} \rightarrow$ event set P , greedily matched to the gold set G (highest IoU first, same channel, $\text{IoU} > 0.3$, each event used once)	$2\text{TP}/(2\text{TP} + \text{FP} + \text{FN})$ over the matched events; an empty list scores 1 on an anomaly-free row and 0 otherwise	Section E.5, Eq. (5)
Lead-lag	$\{\text{"direction": str, "lag_samples": int}\} \rightarrow$ relation $\hat{d} \in \{\text{lead, lag, independent}\}$ and offset $\hat{\ell}$, reduced to $\delta = \hat{\ell} - \ell /L$	$1[\hat{d} = d] \cdot \max(0, 1 - \delta/\tau_\ell)$ with $\tau_\ell = 0.05$; the independent case has no gold offset and reduces to three-way classification	Section E.6, Eq. (6)
ANOMALYXL-MCQ — the raw final message, a $\{\text{"answer": "..."}\}$ object, or a trailing <code>answer:</code>			
Presence	Yes / No $\rightarrow$ selected option $\hat{a}$, by exact string match against the gold option text	$1[\hat{a} = a] \in \{0, 1\}$	Section E.7
Start time	Before / Early / Medium / Late, plus No Anomaly $\rightarrow \hat{a}$	$1[\hat{a} = a] \in \{0, 1\}$	Section E.7
Categorization	Level Shift / Transient Spike / Change in Seasonality / Change in Variance / Change in Trend / No Anomaly. Four of the five kind labels (one dropped at random), plus No Anomaly $\rightarrow \hat{a}$	$1[\hat{a} = a] \in \{0, 1\}$	Section E.7
Magnitude band	$1-3 / 3-10 / 10-30 / 30-100 \sigma$, plus No Anomaly $\rightarrow \hat{a}$	$1[\hat{a} = a] \in \{0, 1\}$	Section E.7
Identification	Options for anomalous channels are two singletons, one pair, a triplet, and None, over three named channels $\rightarrow \hat{a}$	$1[\hat{a} = a] \in \{0, 1\}$	Section E.7
Indicator	Leading / Lagging / Correlated / Uncorrelated / No anomaly $\rightarrow \hat{a}$	$1[\hat{a} = a] \in \{0, 1\}$	Section E.7

Table 9: AnomalyXL metrics across both benchmark formulations. For each task, the output the model is required to produce, the quantity recovered from it by the parser, and the resulting score. Windows are half-open index intervals and IoU is defined in Section E.1, Eq. (1). Gold quantities are unhatted, predictions hatted; L is the recording length. The two regimes are posed over the *same* underlying recordings, so the only difference between them is the scoring function in the third column: continuous and evidence-grounded above, a $\{0, 1\}$ choice indicator below. Per-category numbers are unweighted row means (Section E.8).

E.8 Aggregation.

Per-category numbers are the unweighted mean of the per-row score over the rows of that category, and per-configuration numbers average those rows without reweighting by category size. The all-channels column is therefore a macro average of Eq. (5) over rows, not a micro-F1 pooled over global TP/FP/FN counts, and a $K=1$ row carries the same weight as a $K=3$ row.

F Generalization Set Details

The generalization set samples from real clinical recordings—the Long-Term ECG atrial-fibrillation database (Petrutiu, Sahakian, and Swiryn 2007) and the Sleep-PSG polysomnography corpus (Ghassemi et al. 2018)—and are cast into the AnomalyXL-Localize *classify-with-evidence* schema, so the same parser and scorers used throughout the benchmark apply unchanged. Each subset holds 250 windows: 200 carrying exactly one anomaly and 50 anomaly-free (Table 10).

	LTAf (ECG)	Sleep-PSG
Rate, channels	128 Hz, 2	200 Hz, 4
Windows	250 (200 / 50)	250 (200 / 50)
Anomaly kinds	PVC, APC	Central / Obstr. apnea, hypopnea, RERA
Anomaly	one premature beat	one arousal bout
Gold window	RR bracket	native bout range
Length (s)	60–1017 (med. 332)	60–640 (med. 313)
Length (samples)	7.7k–130k	12k–128k
Gold span (med.)	≈ 1.7 s	≈ 17.7 s
Source records	9 LTAfDB	150 PSG participants

Table 10: Generalization set composition. Window counts are given as (anomalous / anomaly-free). Both subsets are balanced across anomaly kinds; channels are z-normalized per channel.

Worked examples: LTAf ECG. Fig. 7 plots sampled rows from the released LTAf subset at full resolution (thin gray, both channels stacked) with the gold window highlighted and a zoomed inset. The PVC example ($L = 7,931$, 62s) has gold {"kind": "Premature Ventricular Contraction (PVC)", "start": 2127, "end": 2290}; the APC example ($L = 7,718$, 60s) has gold {"kind": "Premature Atrial Contraction (APC)", "start": 6396, "end": 6568}; the no-anomaly example ($L = 9,630$, 75s) has gold {"kind": "No anomaly"}. As in AnomalyXL itself, the gold span is a small fraction of the window, so the full-resolution view is dominated by normal sinus rhythm and the ectopic beat only becomes legible once the inset zooms in.

Worked examples: Sleep-PSG. Fig. 8 plots one sampled row per respiratory-arousal kind from the released Sleep-PSG subset, at full resolution (thin gray, all four channels stacked) with the gold window highlighted and a zoomed inset, each with its gold [start, end) bout window: Central Apnea ($L = 12,396$, gold start=7080, end=9880), Obstructive Apnea ($L = 12,299$, start=5470, end=10490), Hypopnea ($L = 15,299$, start=1300, end=6340), RERA ($L = 12,128$, start=1209, end=4509), and a no-anomaly example ($L = 15,140$).

Window construction. Each positive window is drawn inside a corridor clipped at the nearest neighbouring event, which guarantees that exactly one anomaly falls inside it. For LTAf this is a single premature beat—a premature ventricular (PVC) or atrial (APC) contraction—inside a normal-sinus-rhythm bout containing no other abnormal beat; negatives are clean sinus-rhythm spans. For Sleep-PSG it is a single respiratory arousal—central apnea, obstructive apnea, hypopnea, or a respiratory-effort-related arousal—with no other arousal in the window; negatives are arousal-free, non-wake spans. Within each corridor, the window length itself is drawn uniformly across the full supported range: 60s up to ≈ 17 min for LTAf, 60s up to ≈ 11 min for Sleep-PSG, capped at ≈ 131 k samples. Positive windows are drawn un-

der a fixed per-kind quota (LTAf: PVC and APC each ≈ 100 rows; Sleep-PSG: 50 rows per one of the four arousal kinds) via a round-robin scheduler that alternates across kinds until each quota is met (Table 10). Negative windows are drawn independently from clean gaps of the same length range. The released windows come from two seeded passes (42 for the base draw, 43 for a subsequent long-context extension) for reproducibility. Fig. 9 gives the resulting class balance and context-length distribution: both length histograms are visibly bimodal — a cluster of shorter windows near the 60 s floor and a broad tail of long-context windows reaching the ≈ 131 k-sample cap — so short and long contexts are both well represented rather than concentrated near the median.

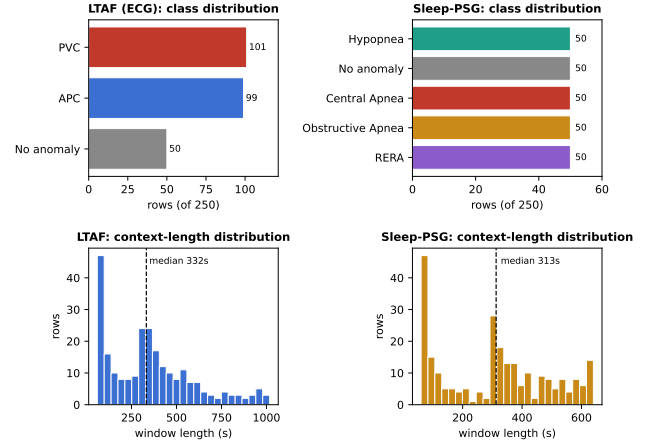


Figure 9: Generalization-set class and context-length distributions, computed directly from the released ltaf-anomaly-v2 and sleep-anomaly-v2 parquet files. Dashed line marks the median window length.

Unified detect-and-classify prompt. A single prompt template is used for both positive and negative windows, so detection and localization are measured under one task rather than a separate present/absent question. The LTAf instantiation reads, verbatim from the released dataset:

LTAf unified detect-and-classify prompt (verbatim, $L = 7,931$)

Determine whether the time series contains an anomaly and, if so, identify it and its window. Respond with exactly one JSON object on a single line, no other text: {"kind": <label>, "start": <int>, "end": <int>}. kind must be one of: "Premature Ventricular Contraction (PVC)", "Premature Atrial Contraction (APC)", or "No anomaly" if the series contains no anomaly. start is inclusive, end is exclusive; if "No anomaly", omit them. Each anomaly kind is defined as follows: a Premature Ventricular Contraction (PVC) is an early, wide and bizarre QRS complex arising from the ventricles... (Here $L = 7931$.)

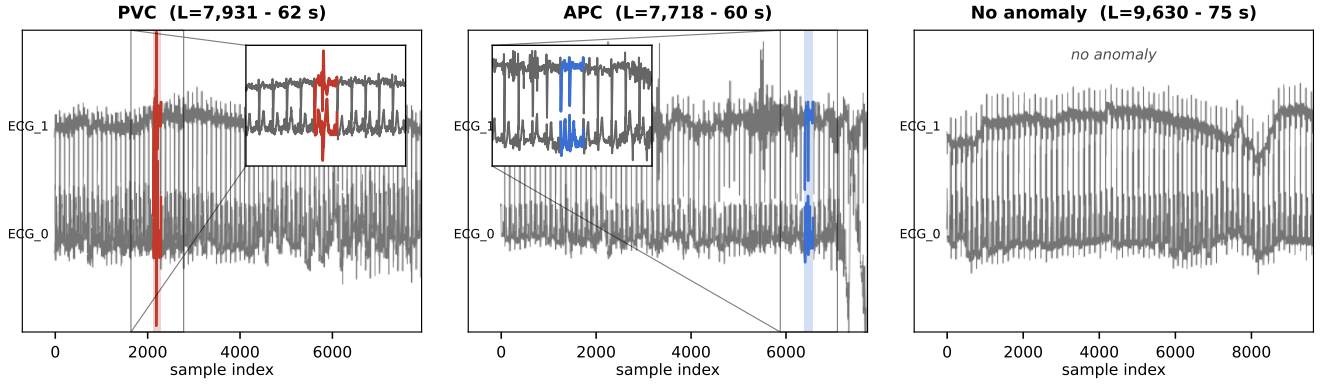


Figure 7: LTAf ECG examples (both channels, ECG_0/ECG_1, stacked): a PVC beat, an APC beat, and a no-anomaly window, sampled from the released `ltaf-anomaly-v2` dataset. Gold windows and exact values are given in the text above.

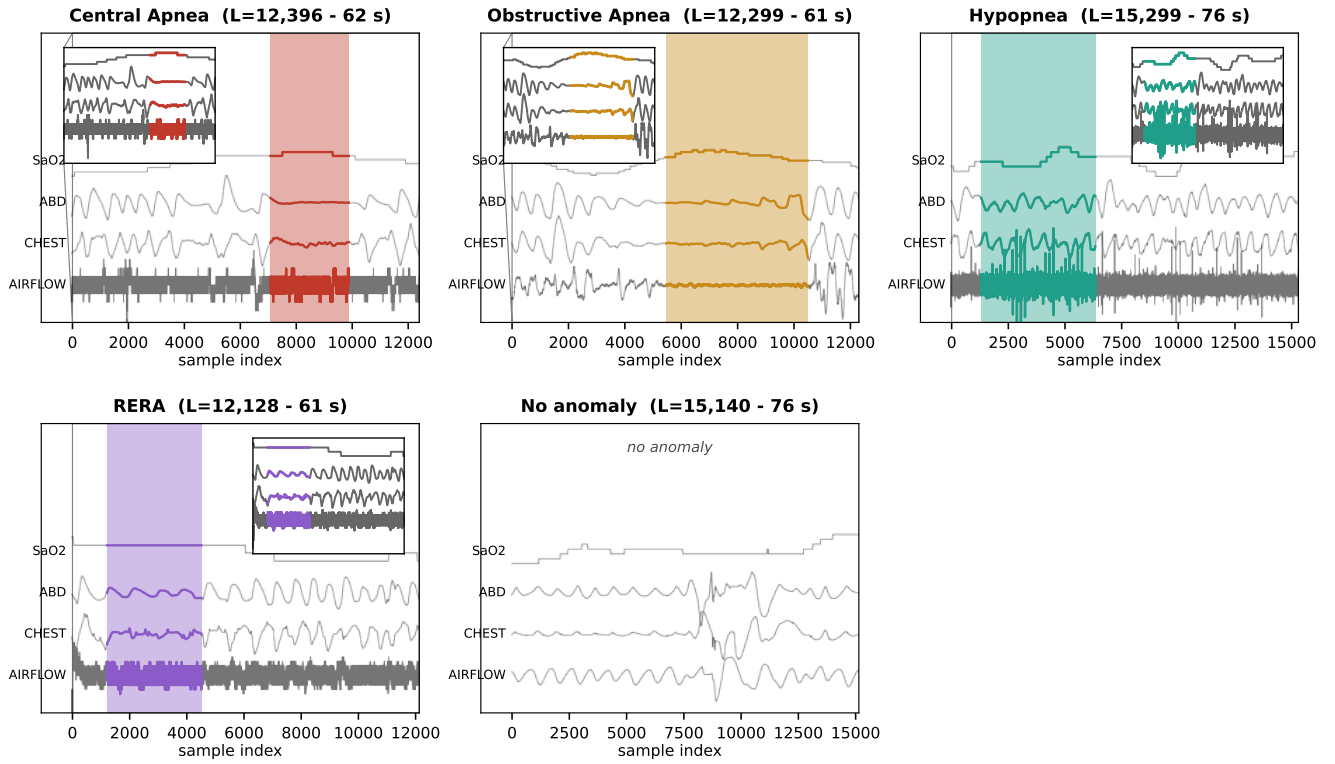


Figure 8: Sleep-PSG examples (four channels, AIRFLOW/CHEST/ABD/SaO2, stacked): one window per respiratory-arousal kind plus a no-anomaly window, sampled from the released `sleep-anomaly-v2` dataset. Gold windows and exact values are given in the text above.

The Sleep-PSG instantiation is identical in structure, substituting the four respiratory-arousal kind labels and their clinical definitions (central apnea, obstructive apnea, hypopnea, RERA) for the ECG beat kinds. Scoring is `kind_x_iou`: kind-match times temporal IoU against the gold window, with a correctly predicted No anomaly credited in full on negatives.